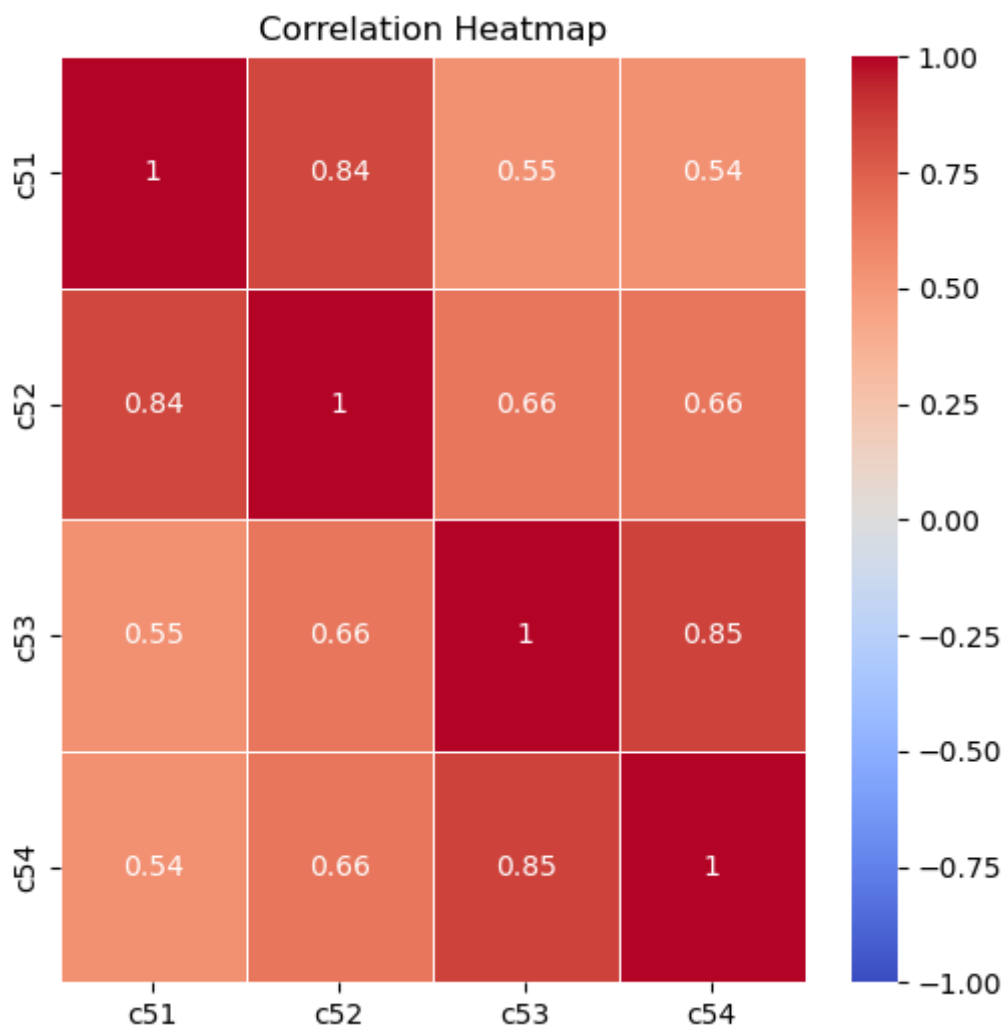
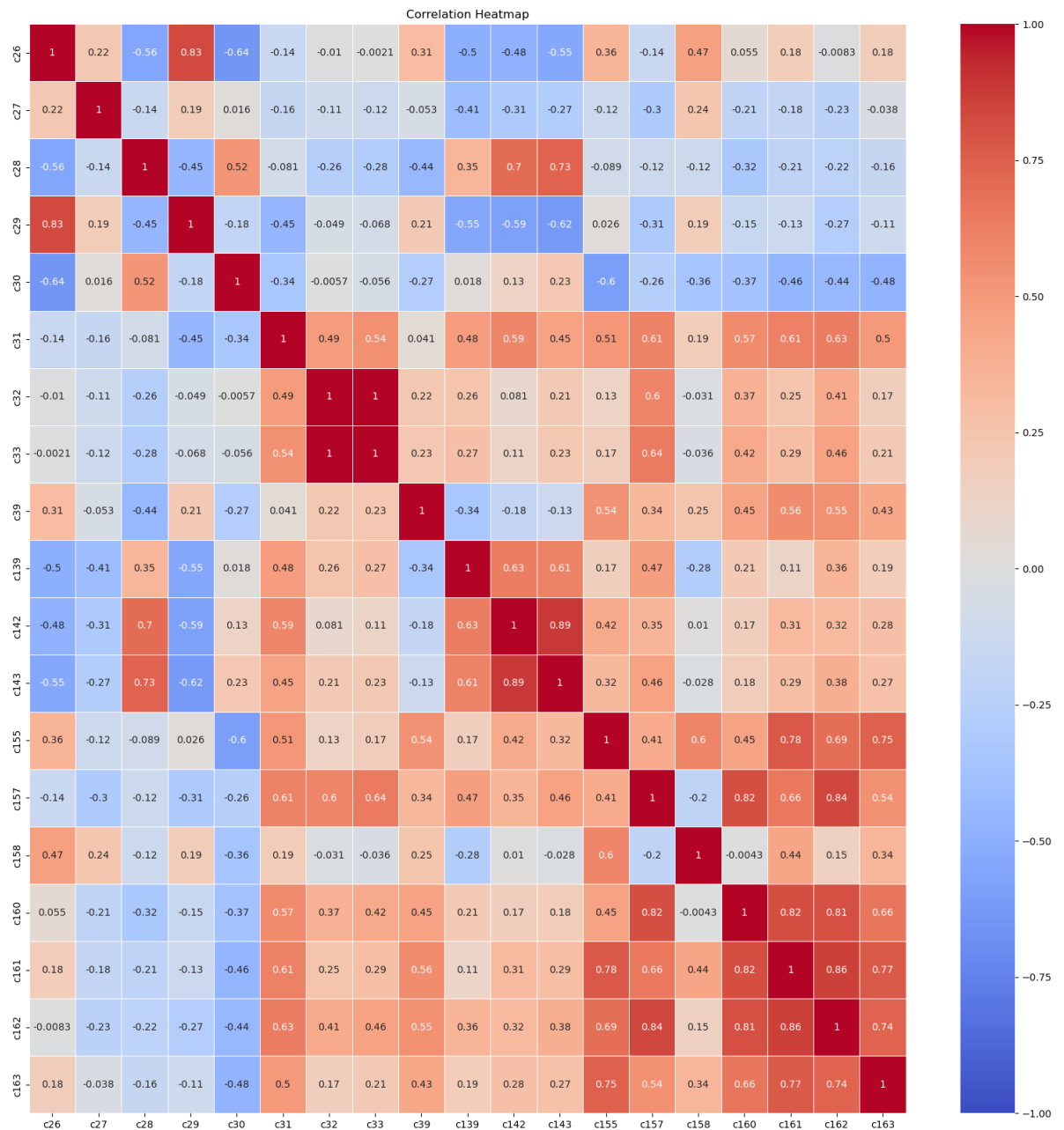


```
In [ ]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df=pd.read_csv('cleaned_e11.csv')
```

```
In [ ]: correlation_matrix = df[['c51', 'c52', 'c53','c54']].corr()
plt.figure(figsize=(6, 6)) # Set the figure size (optional)
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5, vmin=-1, vmax=1)
plt.title('Correlation Heatmap')
plt.show()
```

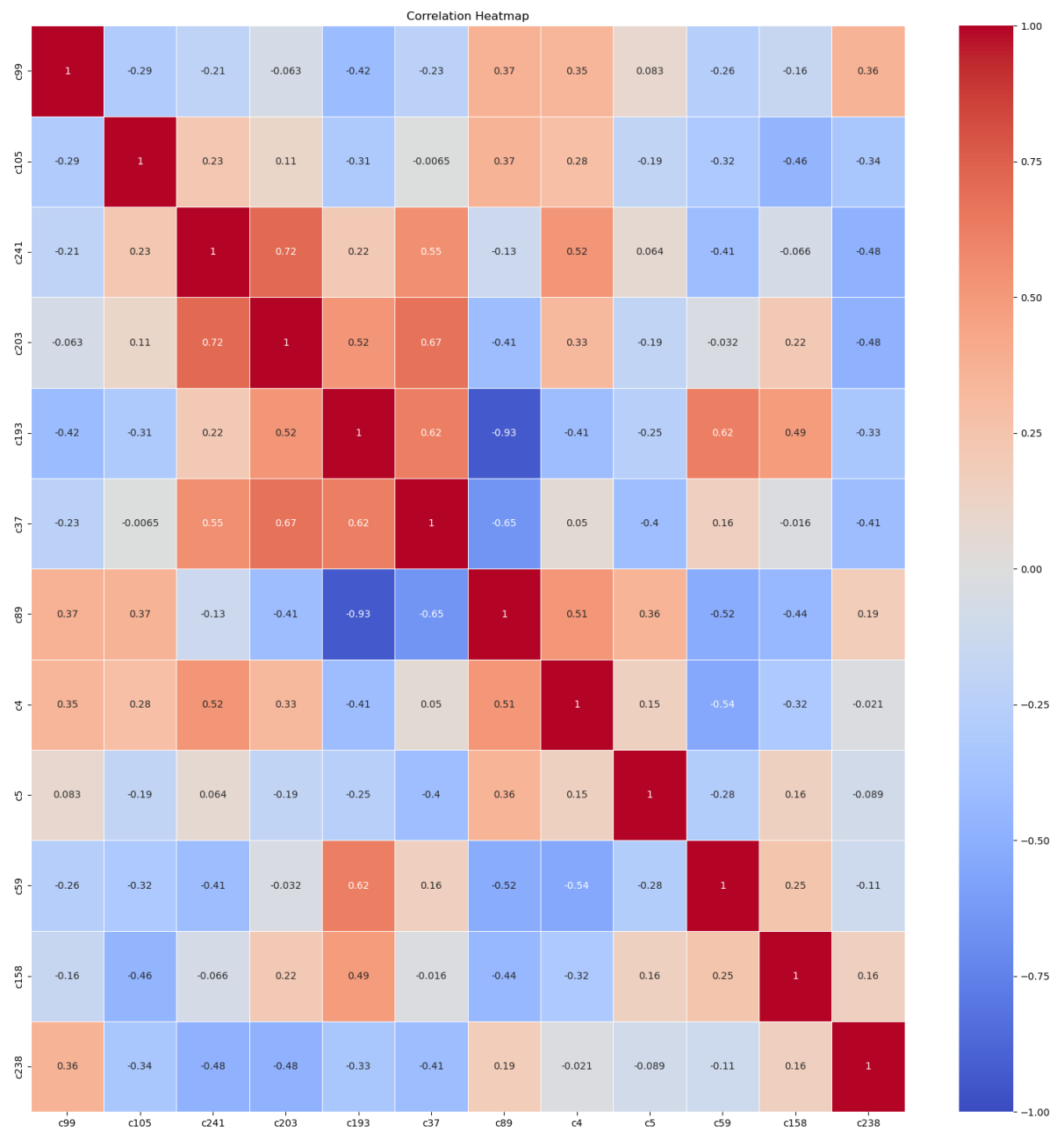


```
In [ ]: correlation_matrix = df[['c26','c27', 'c28', 'c29', 'c30', 'c31', 'c32',
'c33', 'c39', 'c139', 'c142', 'c143', 'c155', 'c157', 'c158', 'c160', 'c161', 'c162',
'c163', 'c164', 'c165', 'c166', 'c167', 'c168', 'c169', 'c170', 'c171', 'c172', 'c173', 'c174', 'c175', 'c176', 'c177', 'c178', 'c179', 'c180', 'c181', 'c182', 'c183', 'c184', 'c185', 'c186', 'c187', 'c188', 'c189', 'c190', 'c191', 'c192', 'c193', 'c194', 'c195', 'c196', 'c197', 'c198', 'c199', 'c200']].corr()
plt.figure(figsize=(20, 20)) # Set the figure size (optional)
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5, vmin=-1, vmax=1)
plt.title('Correlation Heatmap')
plt.show()
```



```
In [ ]: '''columns=[]
for i,c in enumerate(df.columns):
    columns.append(c)'''

correlation_matrix = df[['c99','c105', 'c241', 'c203', 'c193', 'c37','c89','c4','c5
]].corr()
plt.figure(figsize=(20, 20)) # Set the figure size (optional)
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5,vmin=-1
plt.title('Correlation Heatmap')
plt.show()
```



```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [ ]: df = pd.read_csv("D:\IITB\ppt\DS 203\e11.csv")
```

```
In [ ]: df.head()
```

```
Out[ ]:
```

	c1	c2	c3	c4	c5	c6	c7	c8	c9	c10	...	c231	c232	c233	c234	c235	c236	
0	01-09-2018	2	164.154800	155.467568	0.707847	1.227071	2.264965	18.978886	7.561194	0.719497	...	NaN	NaN	NaN	NaN	0.670199532	68.098119	6
1	02-09-2018	2	162.115737	153.645223	0.454140	0.671909	2.260011	18.639050	7.844962	0.702348	...	NaN	NaN	NaN	NaN	0.671396131	68.027155	6
2	03-09-2018	2	153.718388	142.880092	0.634565	0.683161	2.085850	16.671939	6.708392	0.656799	...	NaN	NaN	NaN	NaN	0.672457799	67.502954	6
3	04-09-2018	2	169.214952	157.342985	0.646557	0.722743	2.258069	17.871757	7.239110	0.662394	...	NaN	NaN	NaN	NaN	0.670275002	68.202787	6
4	05-09-2018	2	176.335010	163.307519	0.683941	0.681609	2.252392	18.591802	7.257898	0.689549	...	NaN	NaN	NaN	NaN	0.669760363	68.254780	6

5 rows × 240 columns



```
In [ ]: df = df.drop(columns = ['c188', 'c189', 'c190', 'c199', 'c202', 'c204', 'c223', 'c226', 'c229', 'c231', 'c232', 'c233', 'c234'])
```

```
In [ ]: df.head()
```

Out[]:		c1	c2	c3	c4	c5	c6	c7	c8	c9	c10	...	c225	c227	c228	c230	c23
	01-																
0	09-2018	2	164.154800	155.467568	0.707847	1.227071	2.264965	18.978886	7.561194	0.719497	...	0.142312	2.286091	6.661242	30.321605	0.67019953	
	02-																
1	09-2018	2	162.115737	153.645223	0.454140	0.671909	2.260011	18.639050	7.844962	0.702348	...	0.140988	2.289318	6.667243	30.124571	0.67139613	
	03-																
2	09-2018	2	153.718388	142.880092	0.634565	0.683161	2.085850	16.671939	6.708392	0.656799	...	0.143236	2.318084	6.703458	30.107528	0.67245779	
	04-																
3	09-2018	2	169.214952	157.342985	0.646557	0.722743	2.258069	17.871757	7.239110	0.662394	...	0.139518	2.254376	6.616923	29.852068	0.67027500	
	05-																
4	09-2018	2	176.335010	163.307519	0.683941	0.681609	2.252392	18.591802	7.257898	0.689549	...	0.138316	2.218670	6.555687	29.616839	0.66976036	

5 rows × 227 columns

◀

▶

```

In [ ]: l = ['c14','c82','c110','c113','c133','c149','c206','c207','c208','c209','c210','c211','c212','c213','c214','c215','c216','c217',

In [ ]: for j in l:
          df[j].replace('#REF!',float('NaN'),inplace = True)

In [ ]: df['c82']

```

```
Out[ ]: 0      NaN
        1      NaN
        2      NaN
        3      NaN
        4      NaN
        ...
        1020    25
        1021    25
        1022    25
        1023    25
        1024    25
        Name: c82, Length: 1025, dtype: object
```

```
In [ ]: # defining a function to replace the outliers with NaN
```

```
In [ ]: def handle_outliers_iqr(column):
        column = pd.to_numeric(column, errors='coerce') # 'coerce' will replace non-numeric values with NaN
        Q1 = column.quantile(0.25)
        Q3 = column.quantile(0.75)
        IQR = Q3 - Q1
        lb = Q1 - 1.5 * IQR
        ub = Q3 + 1.5 * IQR

        # Replacing values outside the bounds with NaN
        column_copy = column.copy()
        column_copy[(column_copy < lb) | (column_copy > ub)] = float('NaN')

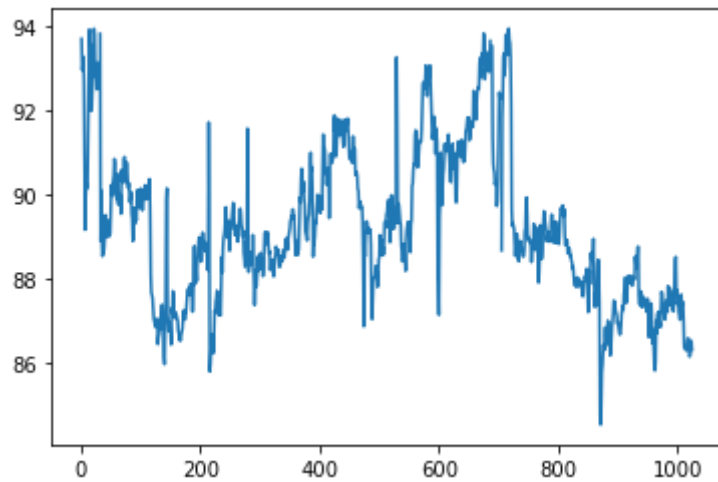
        return column_copy
```

```
In [ ]: numerical_columns = df.columns[82:]
```

```
In [ ]: for column in numerical_columns:
        df[column] = handle_outliers_iqr(df[column])
```

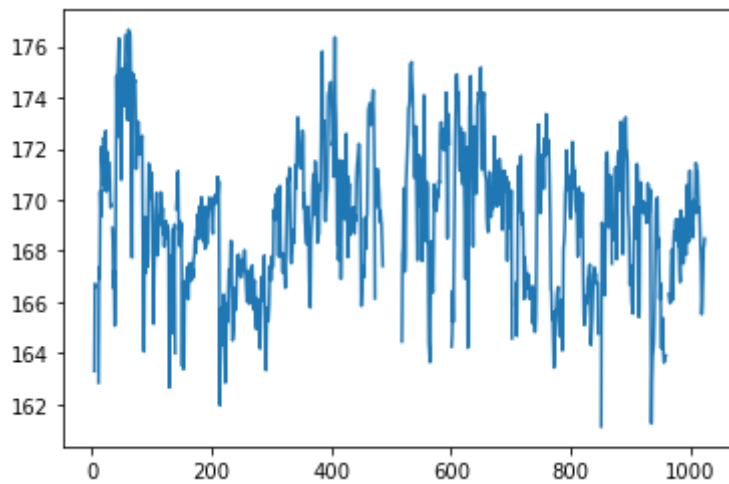
```
In [ ]: plt.plot(df['c83'])
```

```
Out[ ]: [<matplotlib.lines.Line2D at 0x218b2b7f100>]
```



```
In [ ]: plt.plot(df['c4'])
```

```
Out[ ]: [matplotlib.lines.Line2D at 0x218b2c87dc0>]
```



```
In [ ]: numerical_col = df.columns[1:82]
```

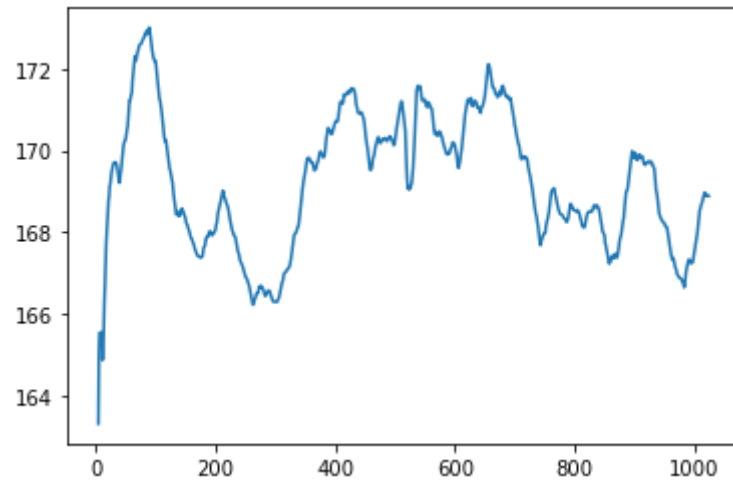
```
In [ ]: for column in numerical_col:  
        df[column] = handle_outliers_iqr(df[column])
```

```
In [ ]: window_size = 50
```

```
In [ ]: nc = df.columns[1:]  
        for column in nc:  
            df[column] = df[column].rolling(window=window_size, min_periods=1).mean()
```

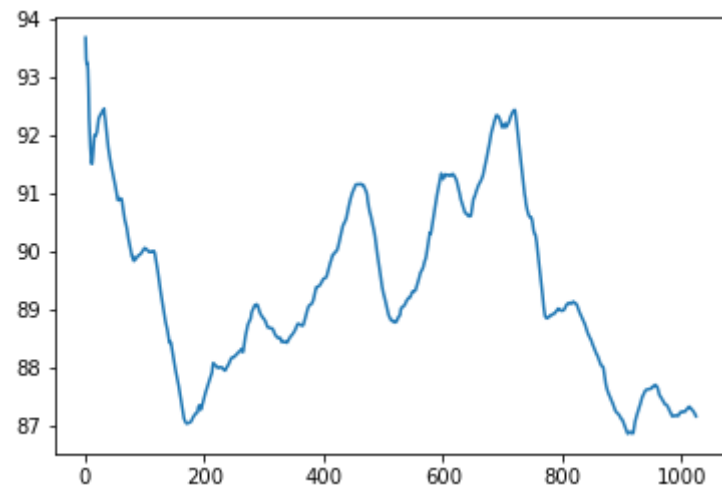
```
In [ ]: plt.plot(df['c4'])
```

```
Out[ ]: [matplotlib.lines.Line2D at 0x218b316b130>]
```



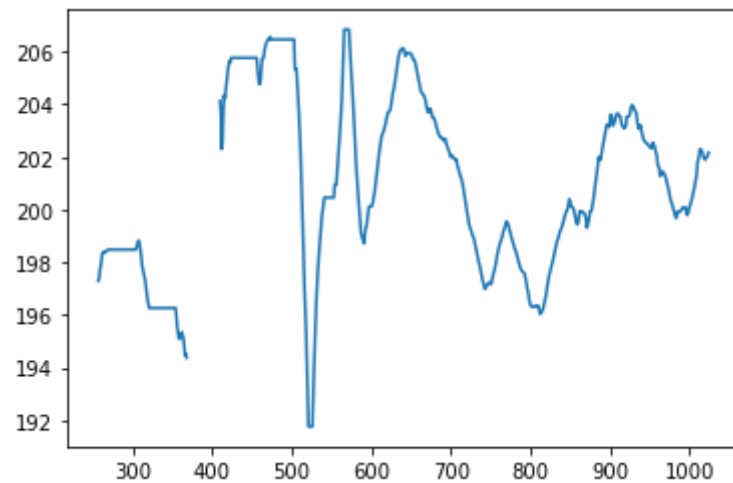
```
In [ ]: plt.plot(df['c83'])
```

```
Out[ ]: [matplotlib.lines.Line2D at 0x218b2c0d670>]
```



```
In [ ]: plt.plot(df['c210'])
```

```
Out[ ]: [<matplotlib.lines.Line2D at 0x218b31ca220>]
```



TO REDUCE NO of MISSING VALUES

```
In [ ]: #nc = frame.columns[1:]
#for column in nc:
#    frame[column] = frame[column].rolling(window=window_size, min_periods=1, center= True).mean()

#frame.to_csv('D:\IITB\ppt\DS 203\E11dataclean2.csv', index=False)
```

```
In [ ]: #c14,c27,c36,c39,c53,c54,c100,c143,c149,c157,c184 columns with issue as they still have some nan values.  
#since values almost constant in most of these cases filling the missing values with mean.  
#c149 is the only
```

```
In [ ]: columns_to_process = ['c14','c27','c39','c53','c54','c100','c143','c157','c184']
```

```
In [ ]: for column in columns_to_process:  
    mean_value = frame[column].mean()  
    frame[column].fillna(mean_value, inplace=True)
```

```
In [ ]: frame['c149'].fillna(0, inplace=True)
```

```
In [ ]: frame['c36'].fillna(frame['c36'].mean(), inplace=True)
```

```
In [ ]: frame = frame.drop(columns = ['c206','c207','c208','c209','c210','c211','c212','c213','c214','c215','c216','c217','c218','c219','
```

```
In [ ]: #just to confirm all missing cells are addressed  
  
nc = frame.columns[1:]  
for column in nc:  
    mean_value = frame[column].mean()  
    frame[column].fillna(mean_value, inplace=True)
```

```
In [ ]: frame.to_csv('D:\IITB\ppt\DS 203\E11datacleanfinal.csv', index=False)
```

```
In [12]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
```

```
In [6]: df=pd.read_csv('transformed_data_e11.csv')
```

```
In [50]: C = ['c237','c186','c185','c171','c162','c159','c157','c155','c153','c131','c127','c14
'''for col in C :
    fig = px.histogram(df[col], x=col, nbins=50,histnorm='probability')
    fig.show()'''
```

```
Out[50]: "for col in C :\n    fig = px.histogram(df[col], x=col, nbins=50,histnorm='probabili
ty')\n    fig.show()"
```

```
In [24]: c={'c237':0.4,'c186':0.3,'c185':0.4,'c157':0.4,'c155':0.45,'c131':0.8,'c127':0.75,'c14
keys=list(c.keys())
```

```
In [28]: for j in keys:
    for i in range (len(df[j])):
        if df.at[i,j]< c[j]:
            df.at[i,j]=0
        else:
            df.at[i,j]=1
```

```
In [37]: for i in range (len(df['c159'])):
    if df.at[i,'c159']< 0.25:
        df.at[i,'c159']= 0
    elif df.at[i,'c159']< 0.7:
        df.at[i,'c159']=1
    else:
        df.at[i,'c159']=2
```

```
In [38]: for i in range (len(df['c153'])):
    if df.at[i,'c153']< 0.2:
        df.at[i,'c153']= 0
    elif df.at[i,'c153']< 0.4:
        df.at[i,'c153']=1
    else:
        df.at[i,'c153']=2
```

```
In [40]: for i in range (len(df['c162'])):
    if df.at[i,'c162']< 0.25:
        df.at[i,'c162']=0
    elif df.at[i,'c162']< 0.6:
        df.at[i,'c162']=1
    elif df.at[i,'c162']< 0.95:
        df.at[i,'c162']=2
    else:
        df.at[i,'c162']=3
```

```
In [46]: df2=pd.read_csv('cleaned_e11.csv')
```

```
In [47]: df[['c51','c52','c53','c54']]=df2[['c51','c52','c53','c54']]  
df['c241']=df2['c241']
```

```
In [49]: df.to_csv('categorised_e11.csv')
```

```
In [ ]:
```

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
dfm = pd.read_csv('cleaned_e11.csv')
```

```
In [1]: # description of columns to decide whether columns are in same scales or not
'''for j in dfm.columns:
    description=dfm[j].describe()
    print(description)
    print("\n")'''
```

```
Out[1]: 'for j in dfm.columns:\n    description=dfm[j].describe()\n    print(description)\n    print("\n")'
```

```
In [ ]: # first 2 output of previous to remove lengthy output we removed output of previous cell

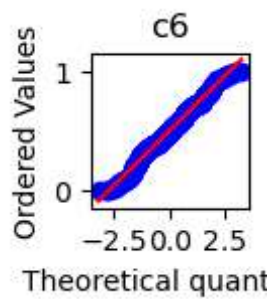
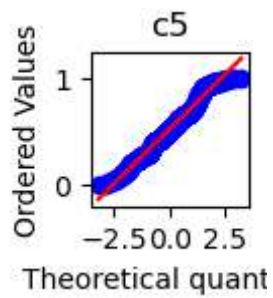
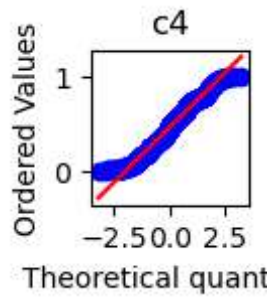
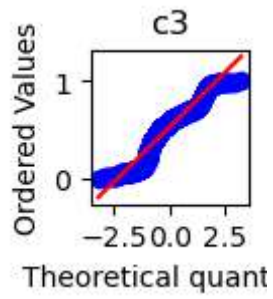
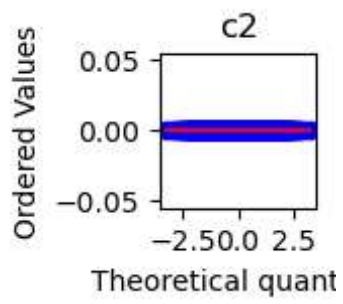
count      1025.0
mean         2.0
std          0.0
min          2.0
25%          2.0
50%          2.0
75%          2.0
max          2.0
Name: c2, dtype: float64

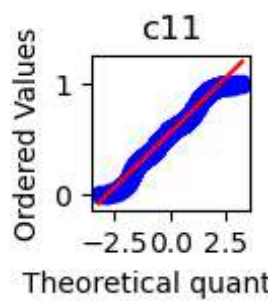
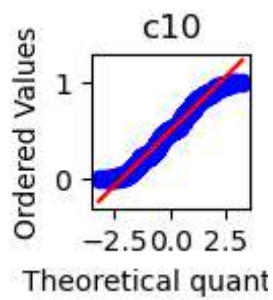
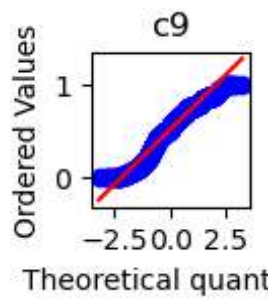
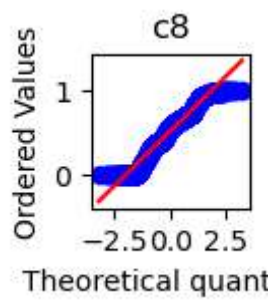
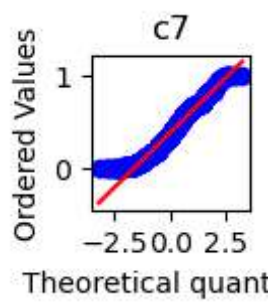
count      1025.000000
mean       171.277209
std         6.932186
min        155.687029
25%        167.825592
50%        172.817589
75%        175.574974
max        185.297196
Name: c3, dtype: float64
```

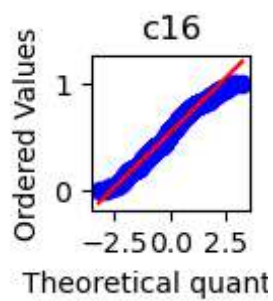
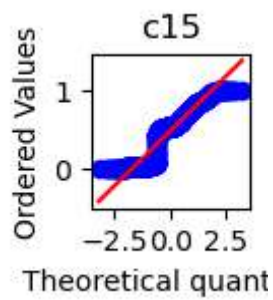
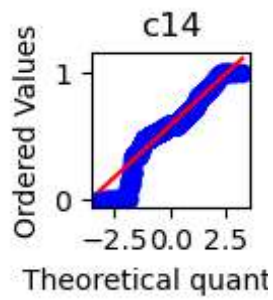
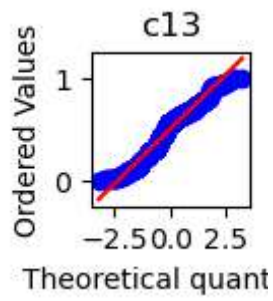
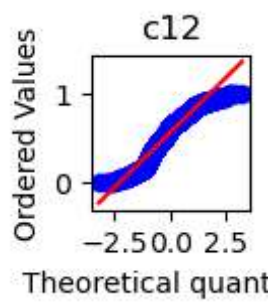
```
In [3]: # normalisation of data
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

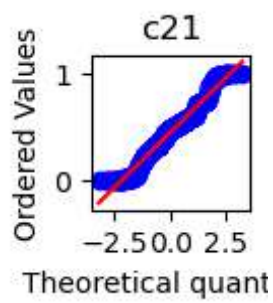
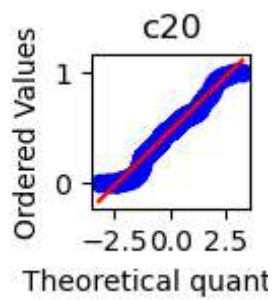
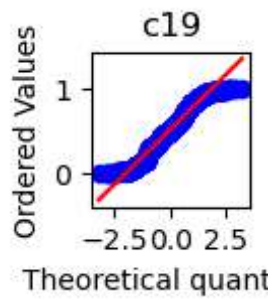
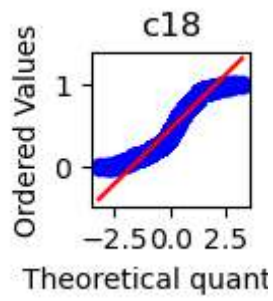
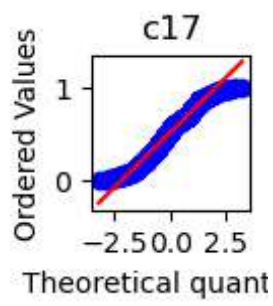
data = pd.read_csv('cleaned_e11.csv')
data=data.drop(['c1','c51','c52','c53','c54'],axis=1)
scaler = MinMaxScaler()
normalized_data = pd.DataFrame(scaler.fit_transform(data), columns=data.columns)
```

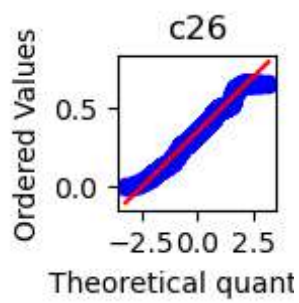
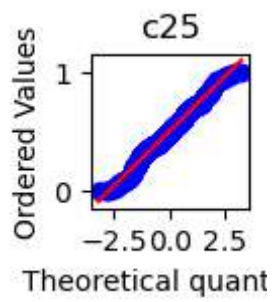
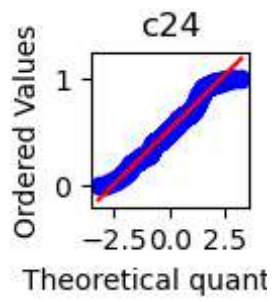
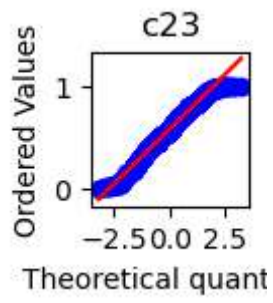
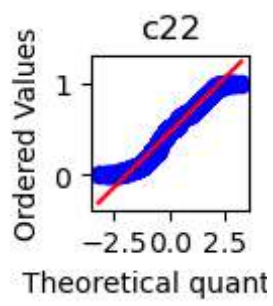
```
In [9]: '''Q-Q plots of normalised to see which columns are right or left skewed so that we can see if
columns data transformation is needed'''
from scipy import stats
dfm=normalized_data
for i in range (len(dfm.columns)):
    fig = plt.figure(figsize=(1,1))
    res = stats.probplot(dfm.iloc[:,i], plot=plt)
    plt.title(dfm.columns.values[i])
    plt.show()
```

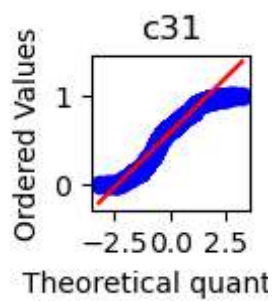
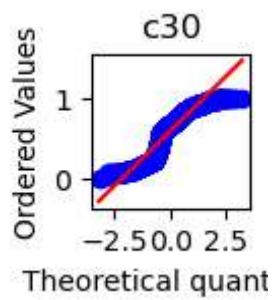
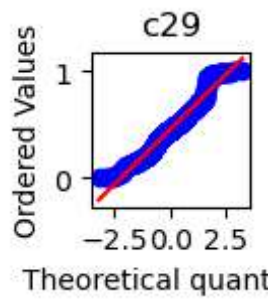
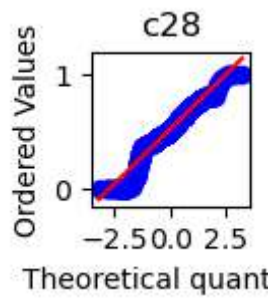
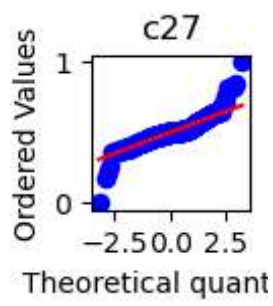


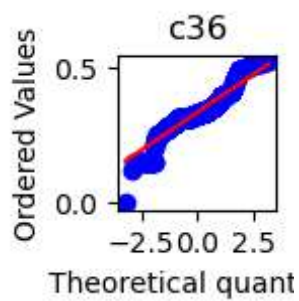
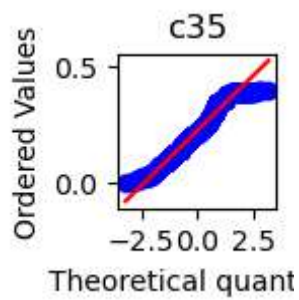
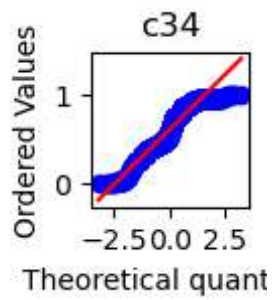
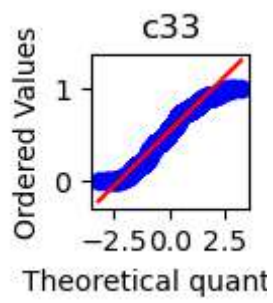
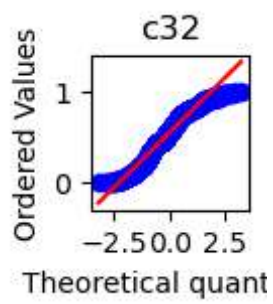


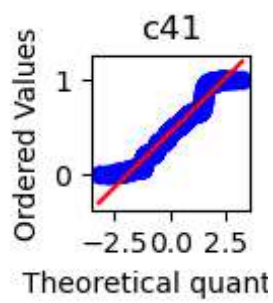
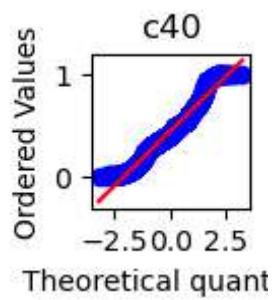
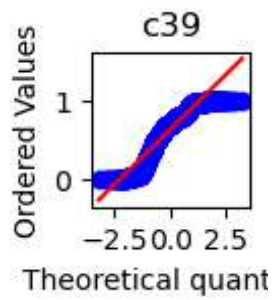
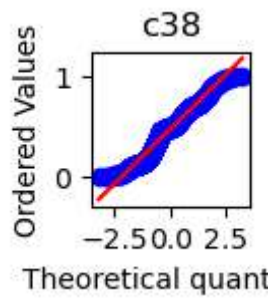
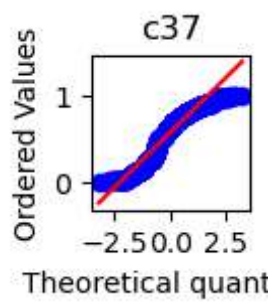


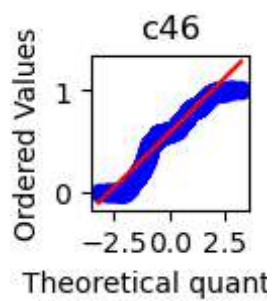
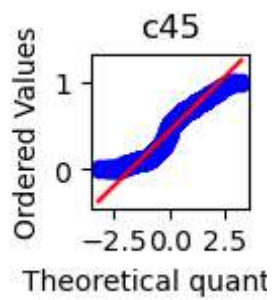
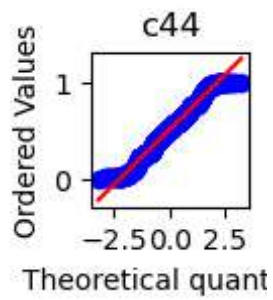
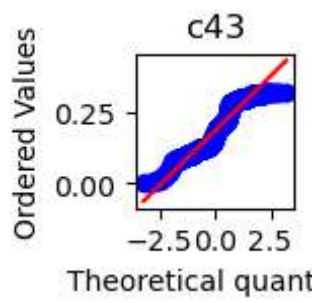
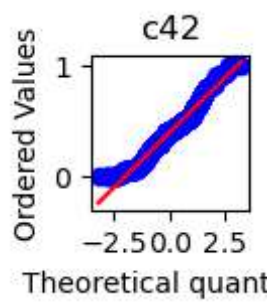


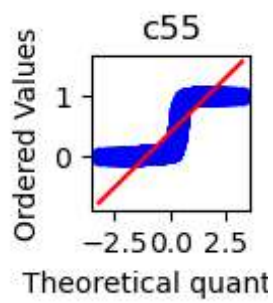
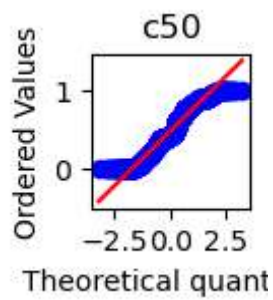
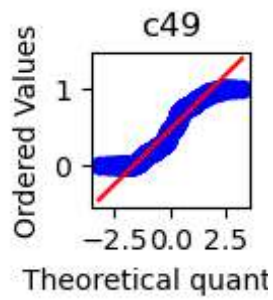
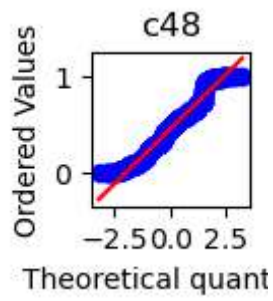
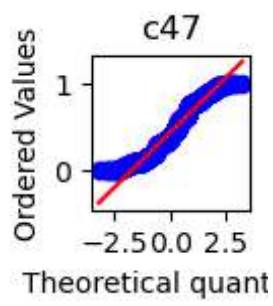


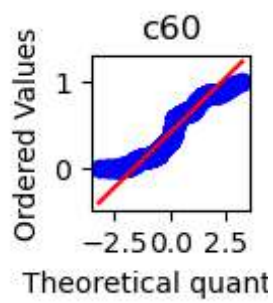
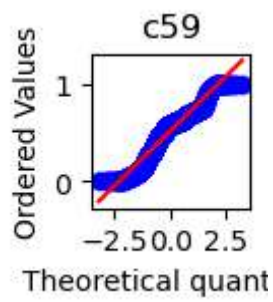
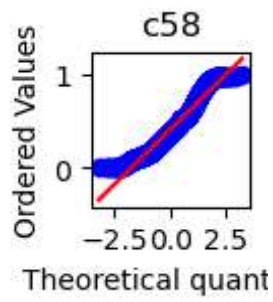
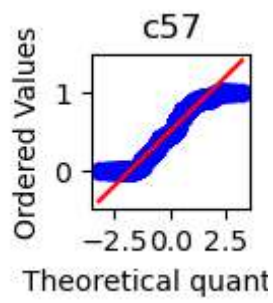
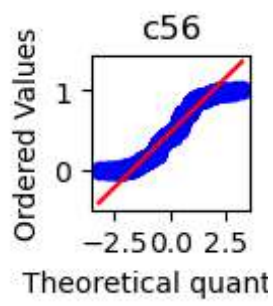


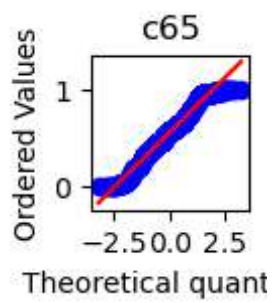
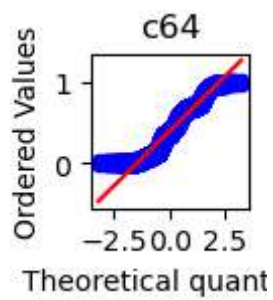
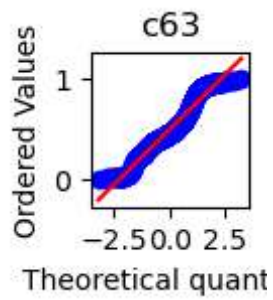
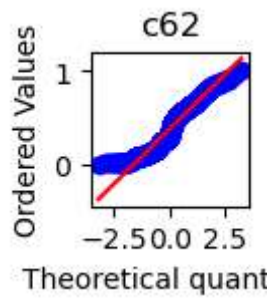
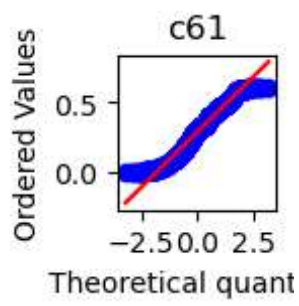


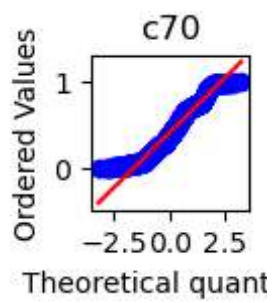
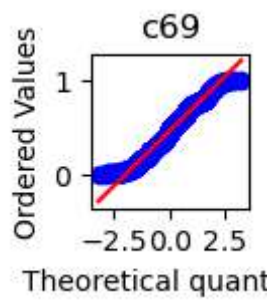
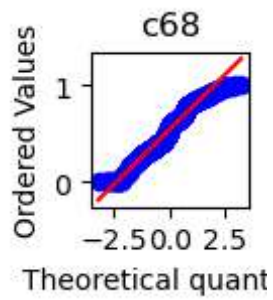
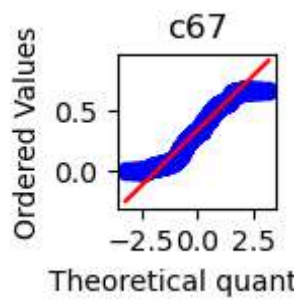
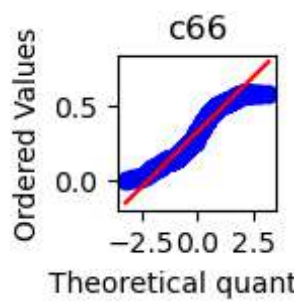


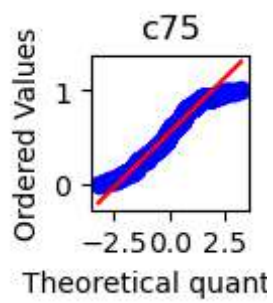
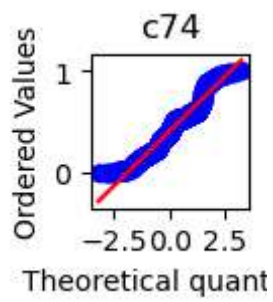
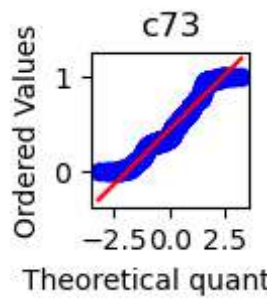
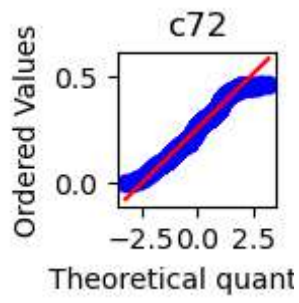
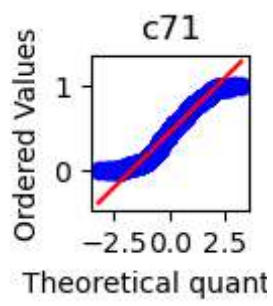


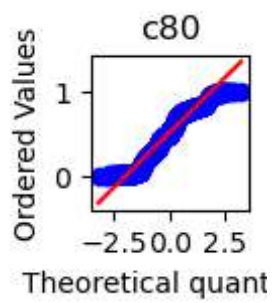
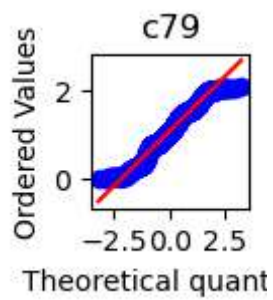
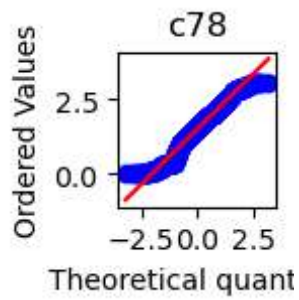
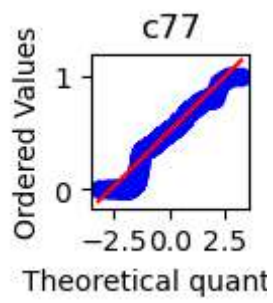
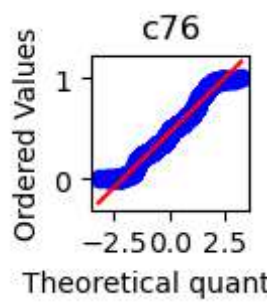


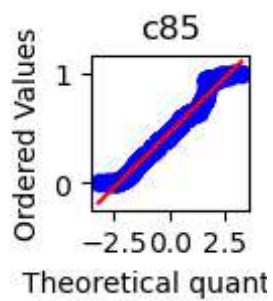
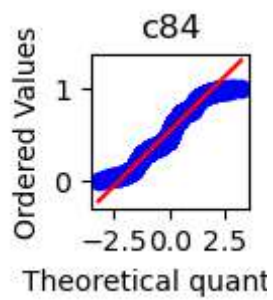
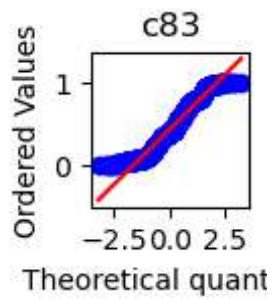
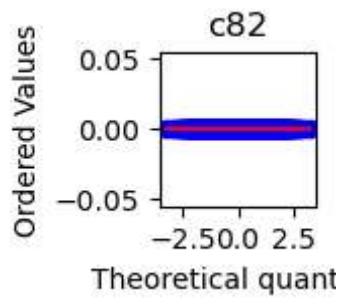
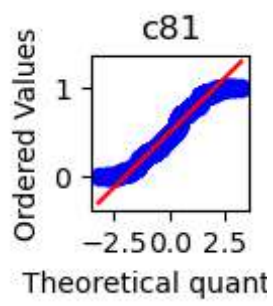


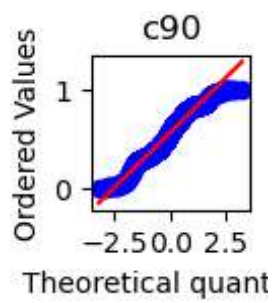
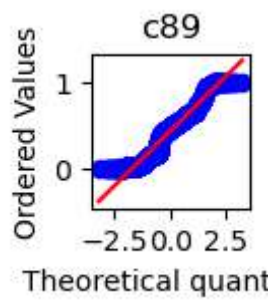
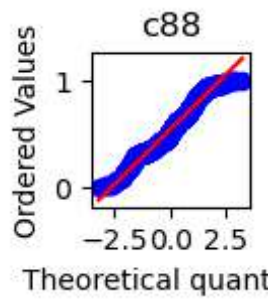
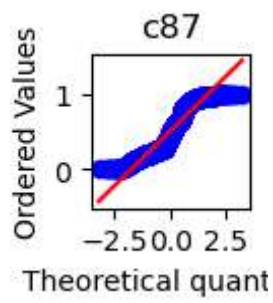
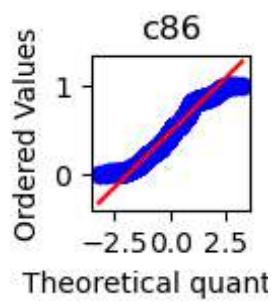


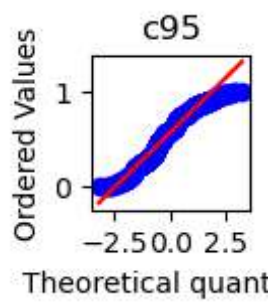
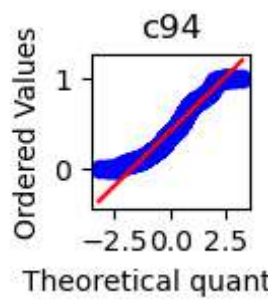
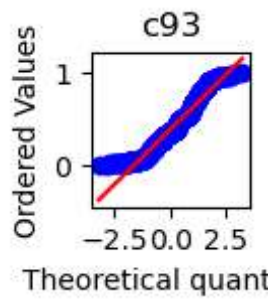
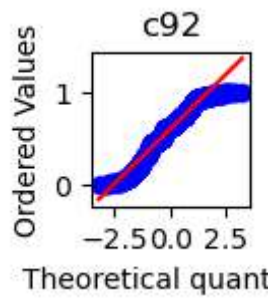
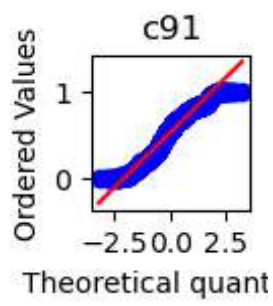


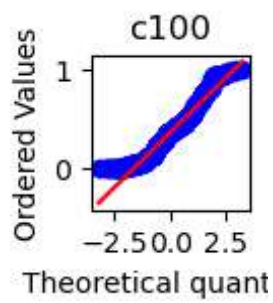
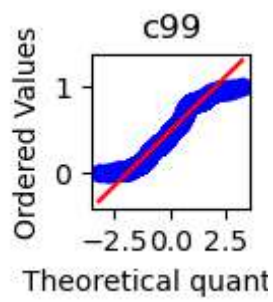
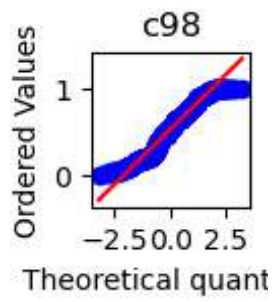
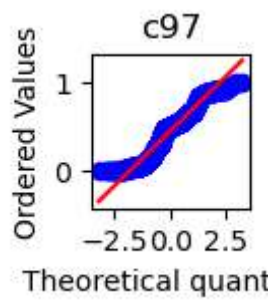
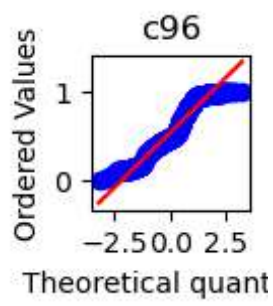


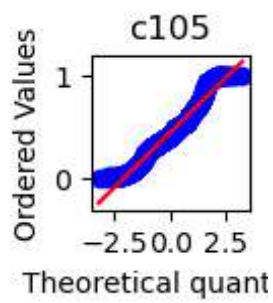
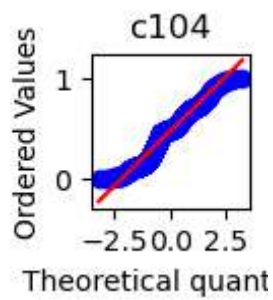
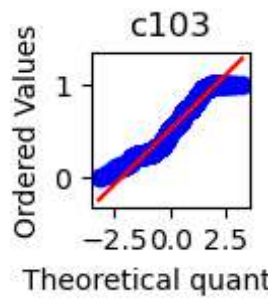
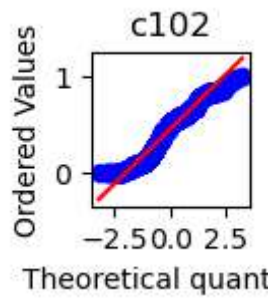
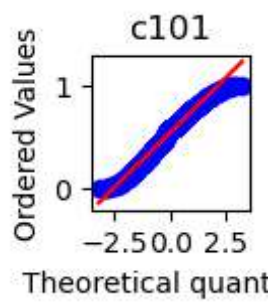


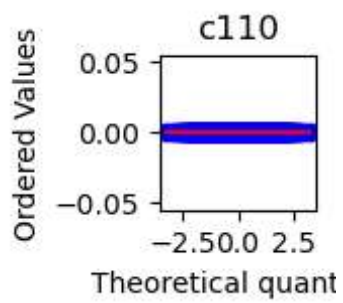
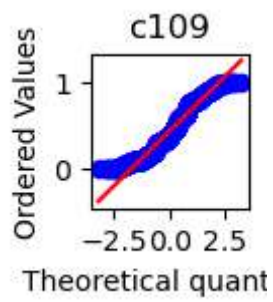
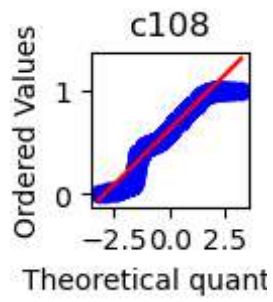
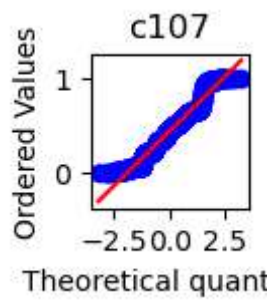
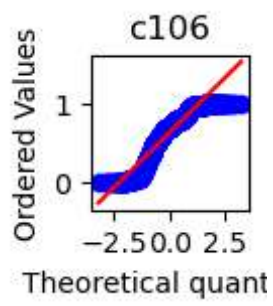


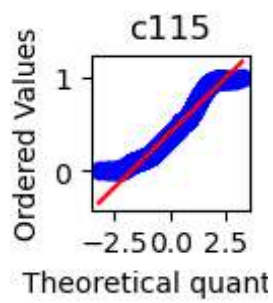
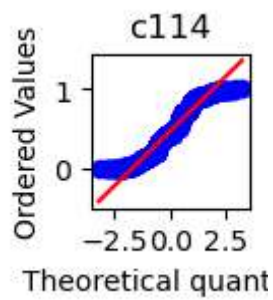
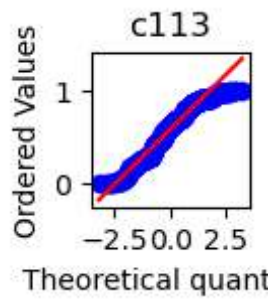
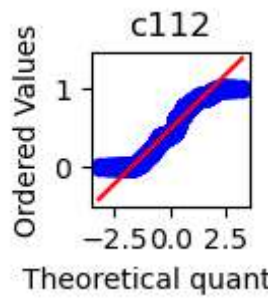
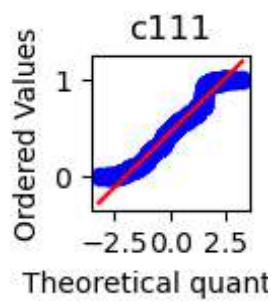


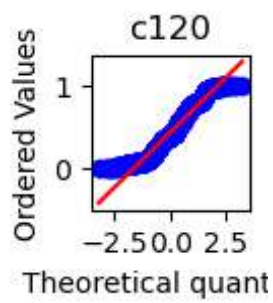
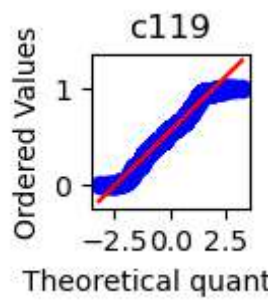
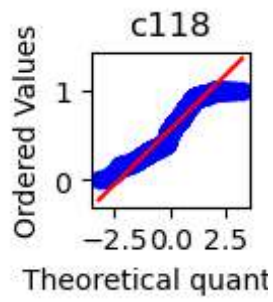
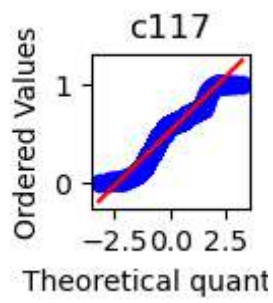
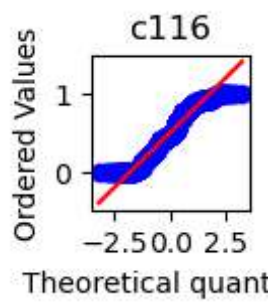


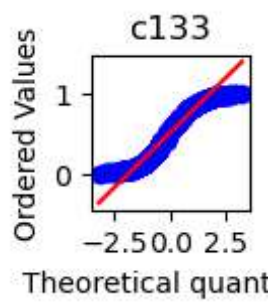
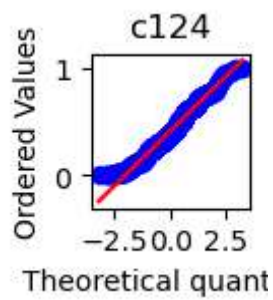
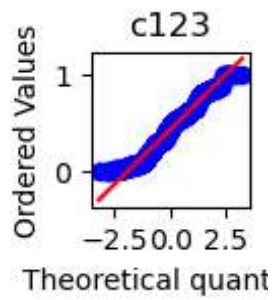
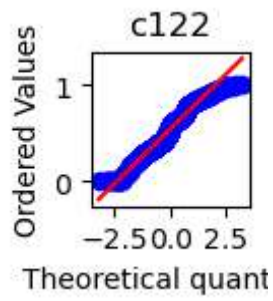
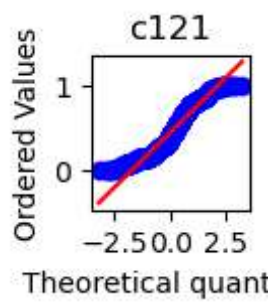


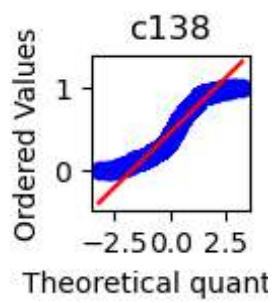
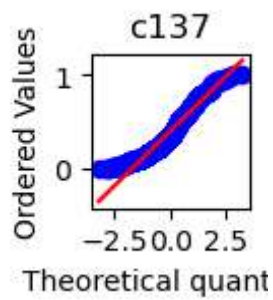
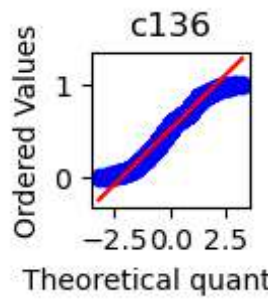
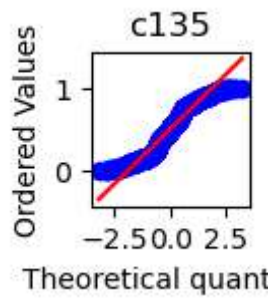
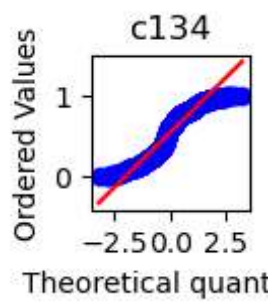


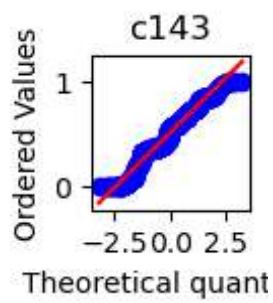
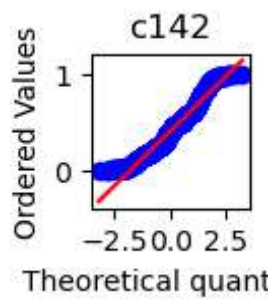
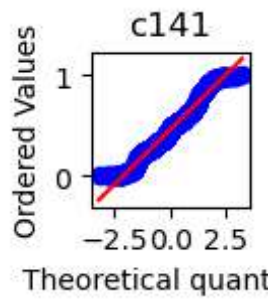
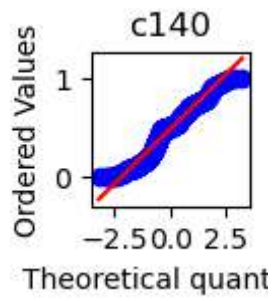
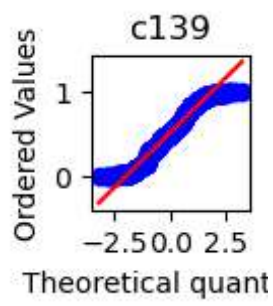


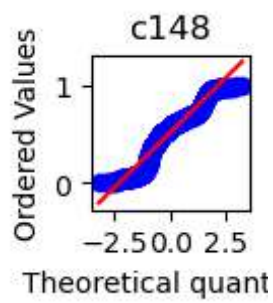
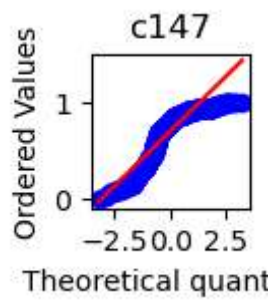
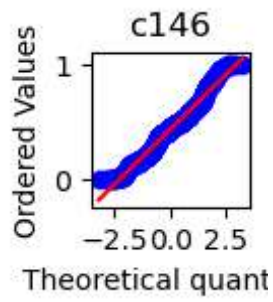
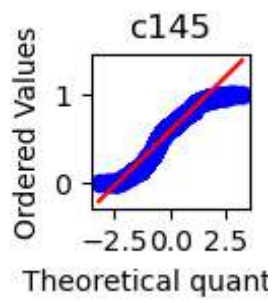
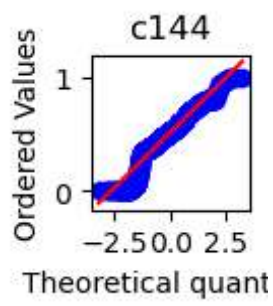


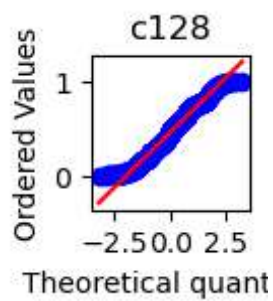
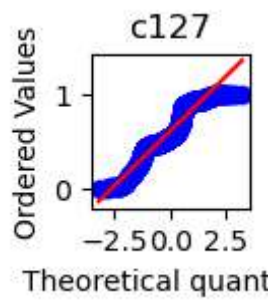
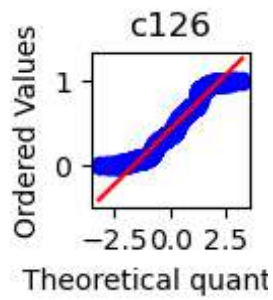
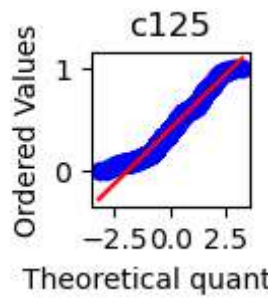
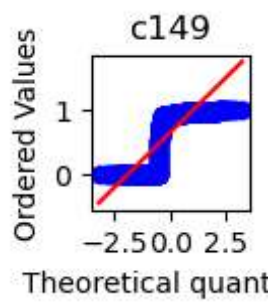


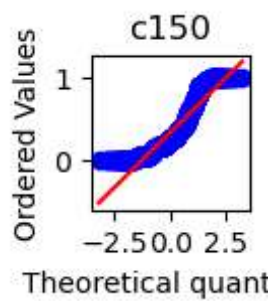
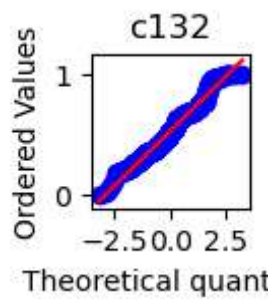
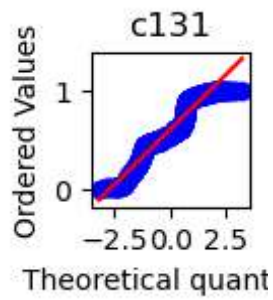
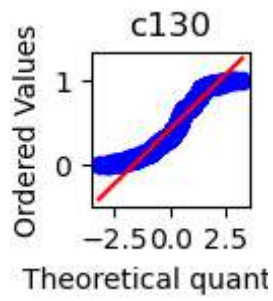
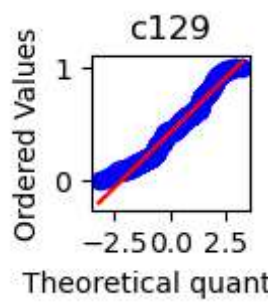


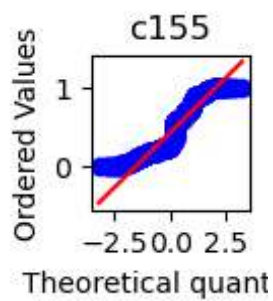
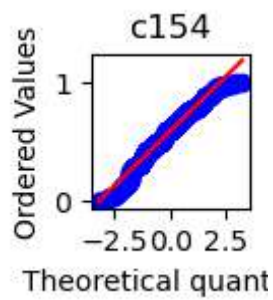
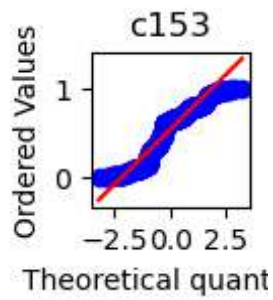
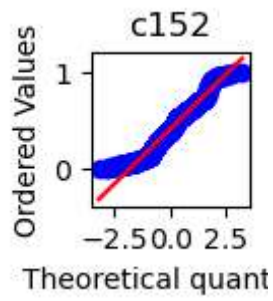
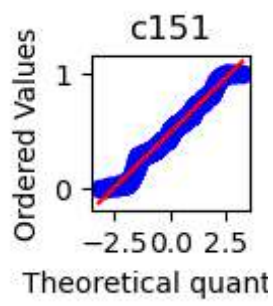


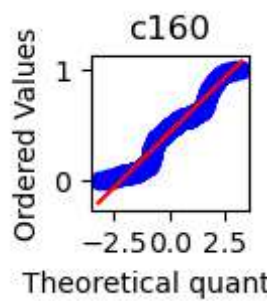
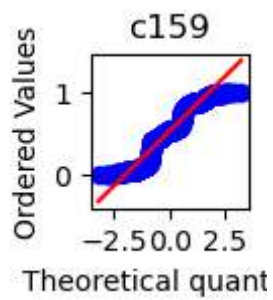
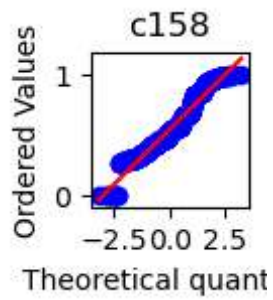
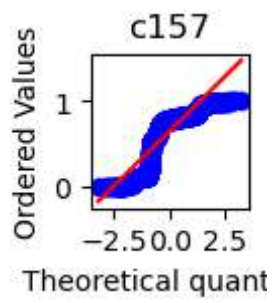
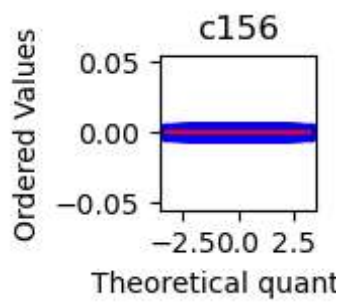


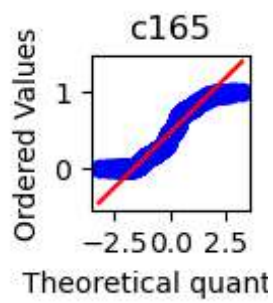
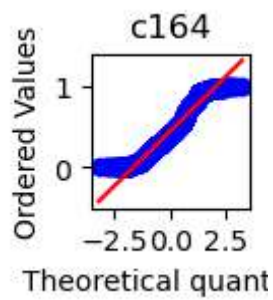
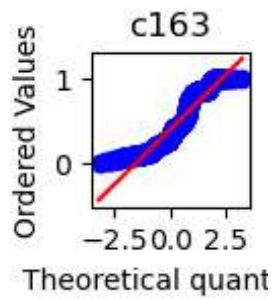
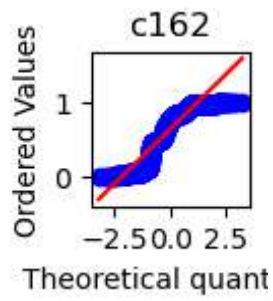
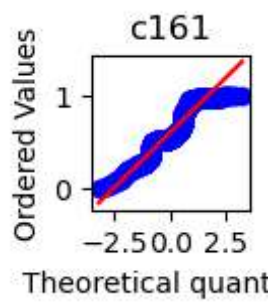


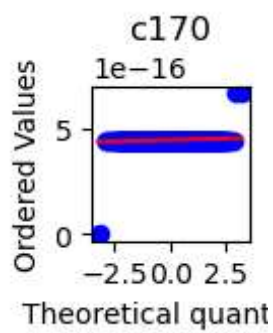
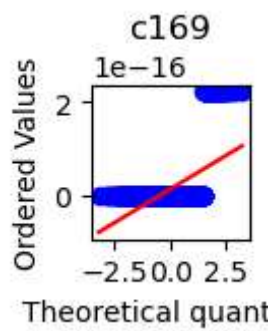
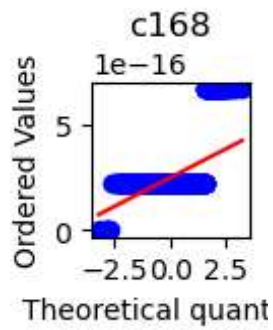
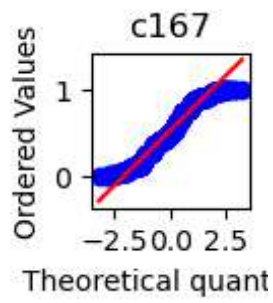
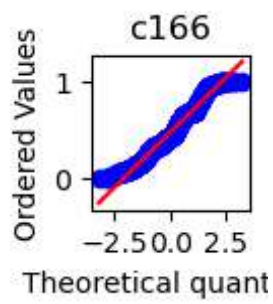


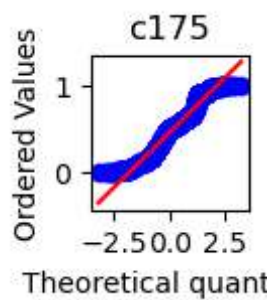
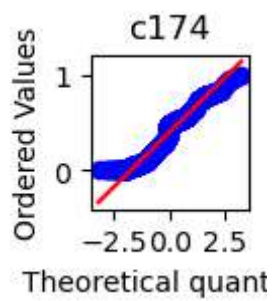
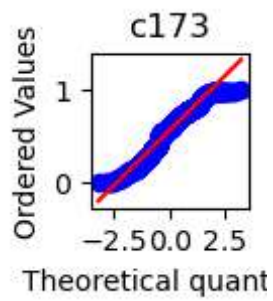
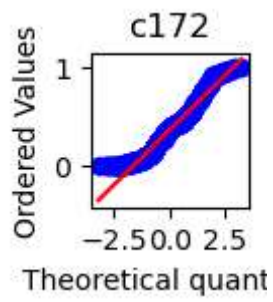
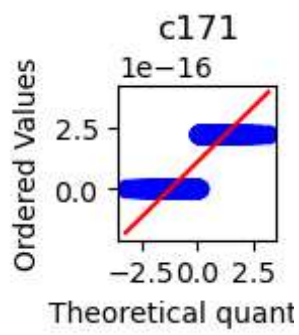


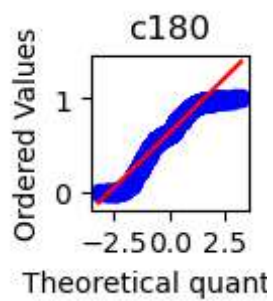
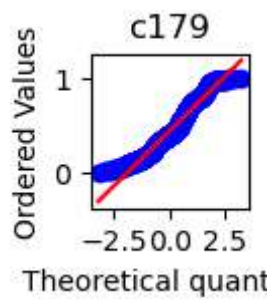
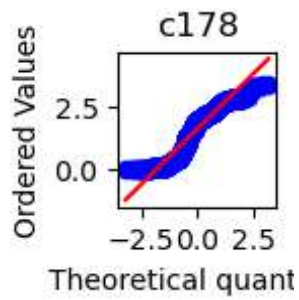
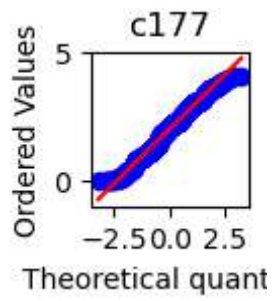
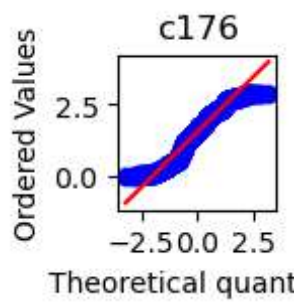


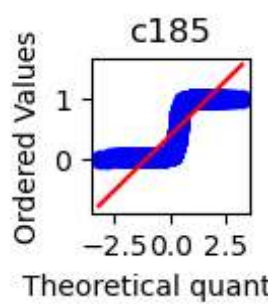
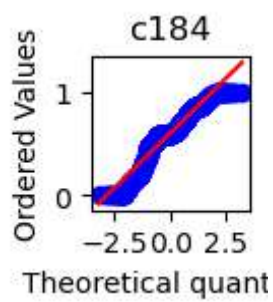
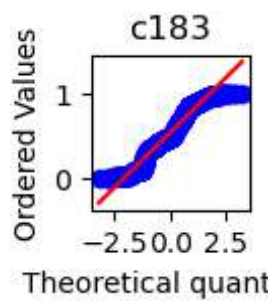
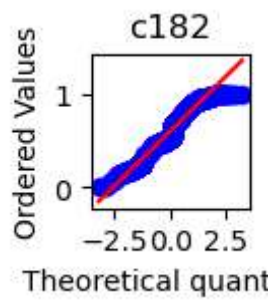
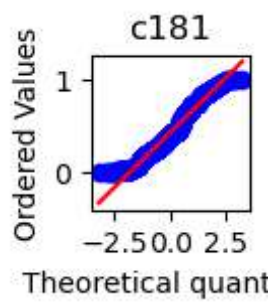


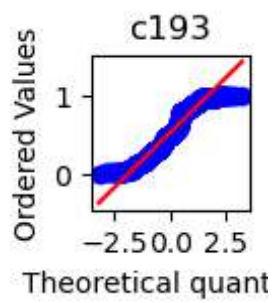
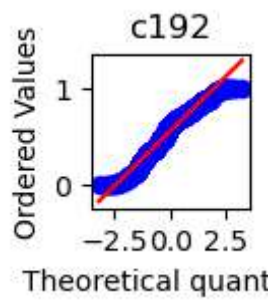
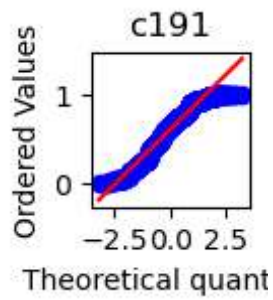
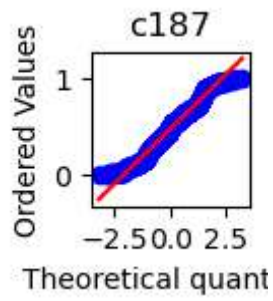
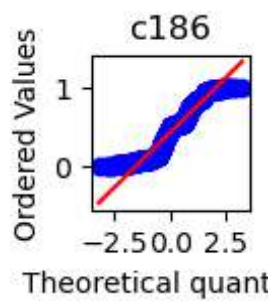


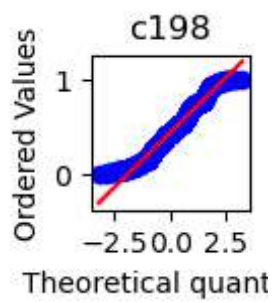
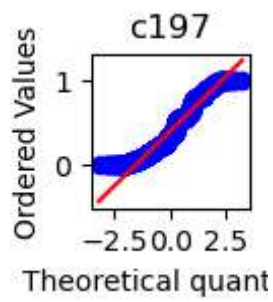
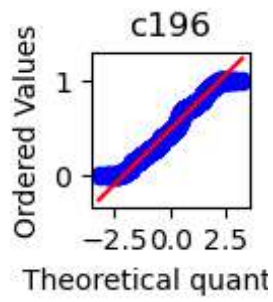
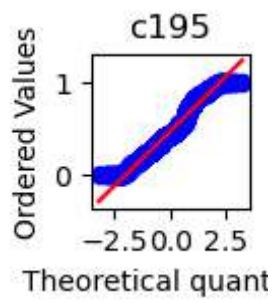
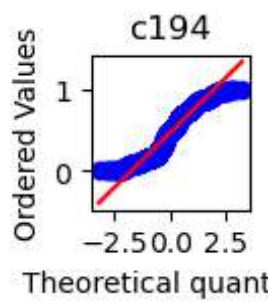


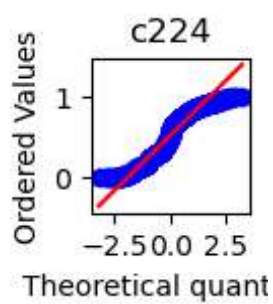
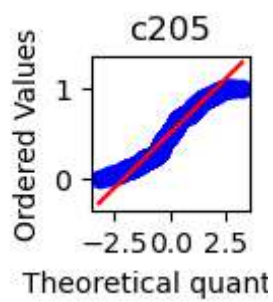
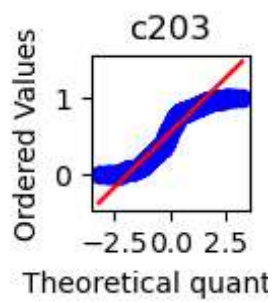
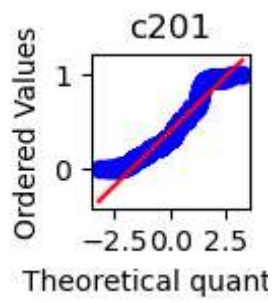
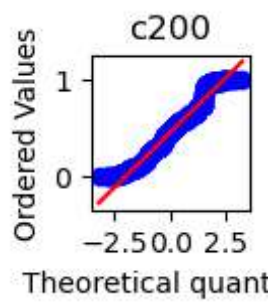


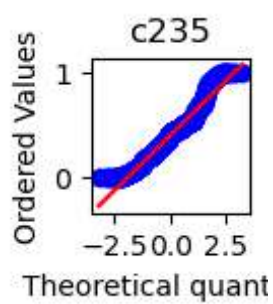
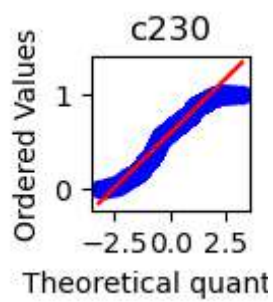
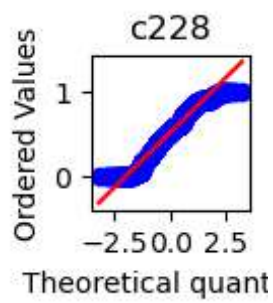
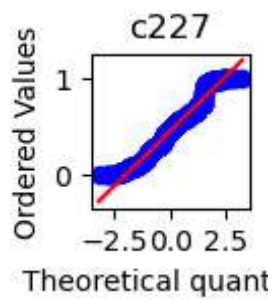
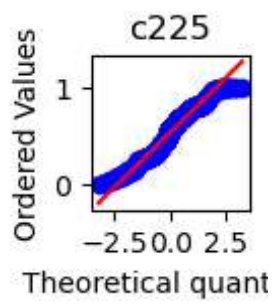


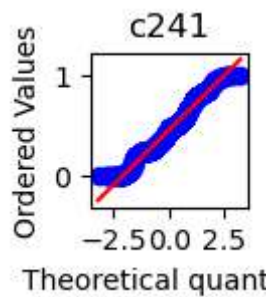
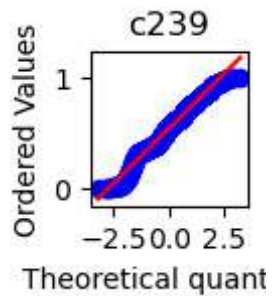
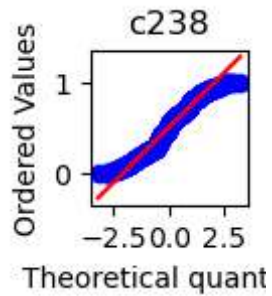
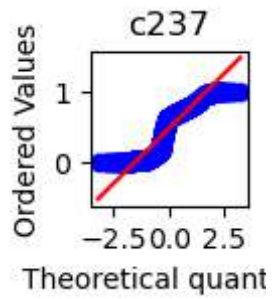
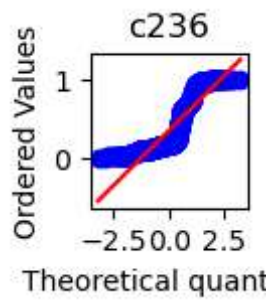












```
In [6]: # box cox method applied to skewed columns and lambda values of transformations listed
from scipy.stats import boxcox

transformed_data = pd.DataFrame()

skewed_columns=['c26','c35','c36','c43','c61','c66','c67','c72','c78','c79','c176','c1
for column in skewed_columns:
    transformed_column, lambda_ = boxcox(dfm[column] + 1)
```

```
transformed_data[column] = transformed_column
print(f"{column} {lambda_}\n")
```

```
for c in skewed_columns:
    dfm[c]=transformed_data[c]
```

c26 -0.1769077488503996

c35 -1.8314592873452558

c36 -0.8601498348159122

c43 -2.6184990285511236

c61 -0.3930774289645634

c66 -0.5388021610221353

c67 -0.09184975276444919

c72 -1.2651944845709733

c78 3.5987247225748678

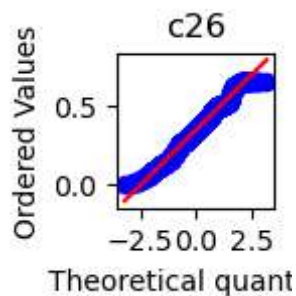
c79 2.7227831105677684

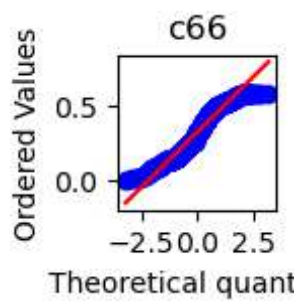
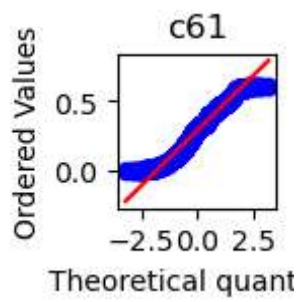
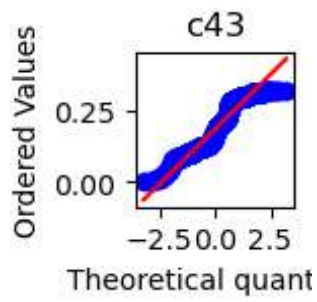
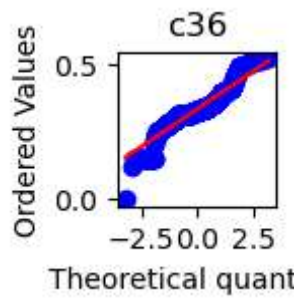
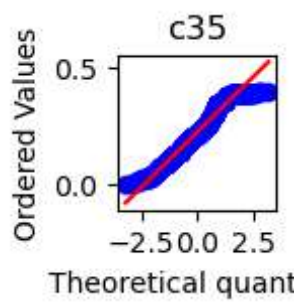
c176 3.4797126310710547

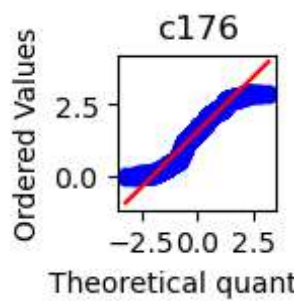
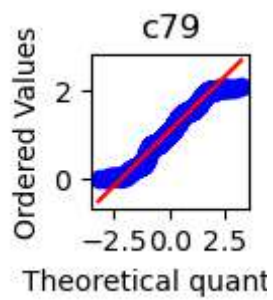
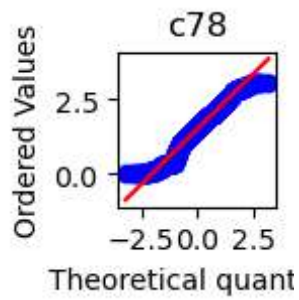
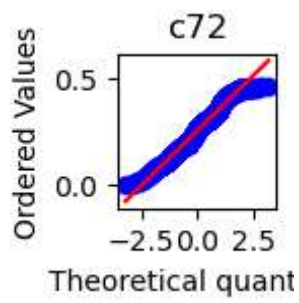
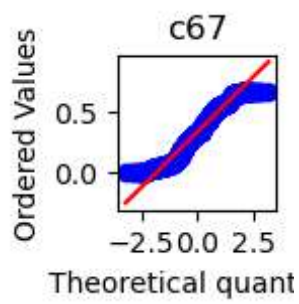
c177 4.174330272382037

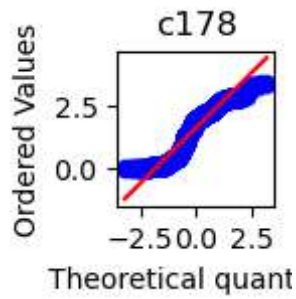
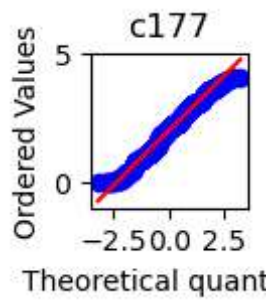
c178 3.777665226779685

```
In [10]: # Q-Q plots of transformed skewed columns to analyse the effectiveness of transformati
dft=transformed_data
for c in (skewed_columns):
    fig = plt.figure(figsize=(1,1))
    res = stats.probplot(dft[c], plot=plt)
    plt.title(c)
    plt.show()
```



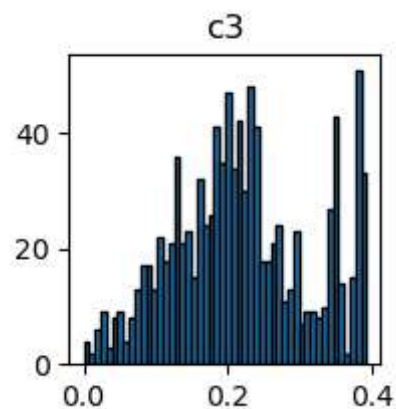
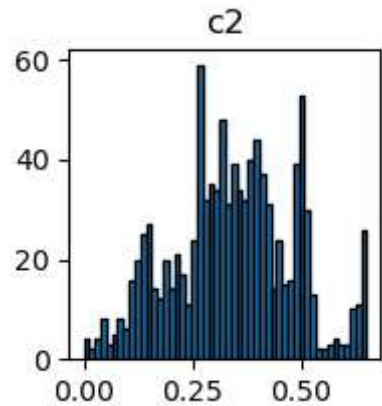


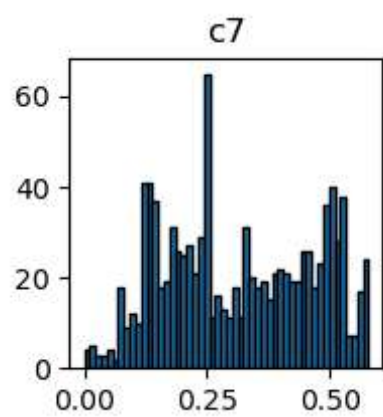
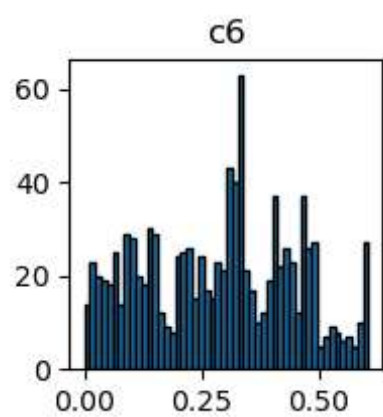
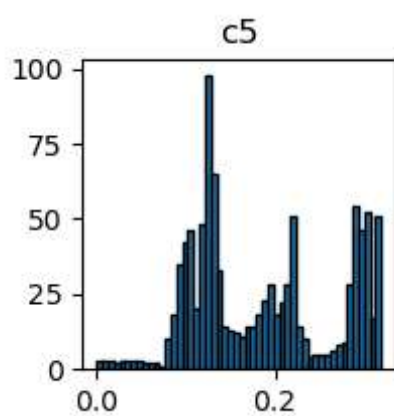
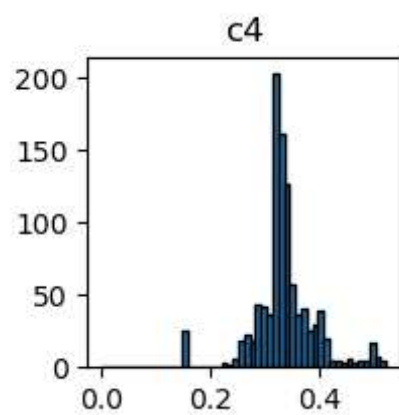


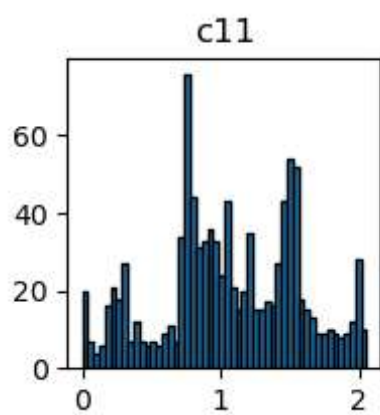
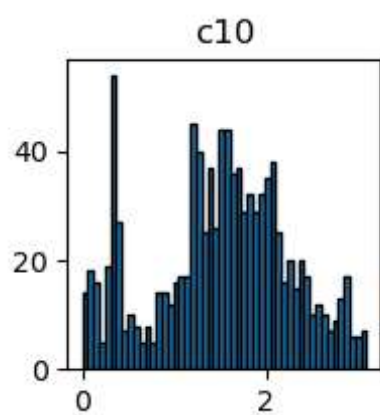
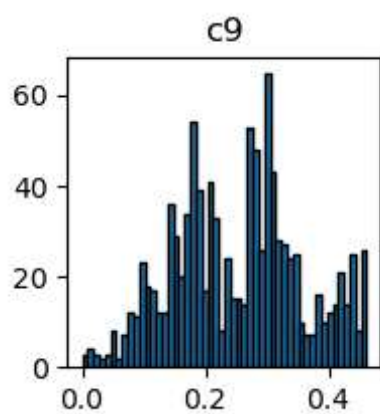
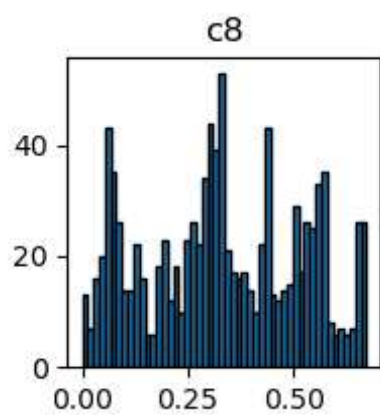


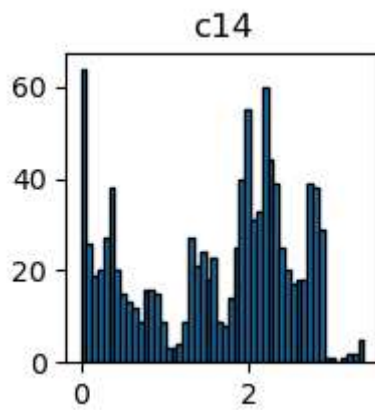
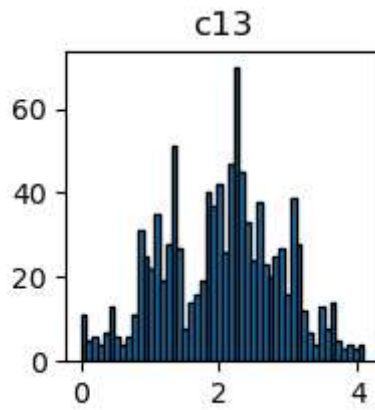
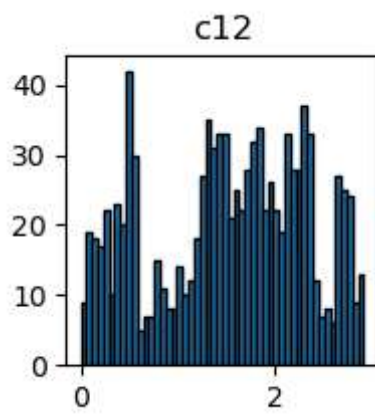
```
In [8]: # histograms of all columns after transformation
for i in range(len(dft.columns)):
    fig = plt.figure(figsize=(1,1))
    hs=plt.hist(dft.iloc[:,i], edgecolor='black', bins=50)
    plt.title(dfm.columns.values[i])
    plt.show()

dfm.to_csv('transformed_data_e11.csv',index=False)
```









```
In [9]: dfm.to_csv('transformed_data_e11.csv',index=False)
```

```
In [ ]:
```